

Acessar

arineto1.github.io/ifpe2026

Informática Teórica

Revisão - Aula 3

Introdução aos Autômatos Finitos Determinísticos (AFD)

Definição Técnica

O **Autômato Finito** é o modelo de computação **mais simples e fundamental** na Teoria da Computação.

Segundo Sipser (2007), estes modelos são ideais **para computadores com uma quantidade extremamente pequena de memória**, focando-se no reconhecimento de padrões em sequências de entrada.

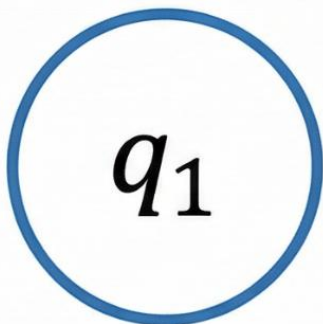
Hopcroft (2001) descreve-os como sistemas que se movem através de um **conjunto finito de estados em resposta a estímulos externos** (símbolos de um alfabeto).



O Alfabeto Visual



Estado (Q - Quê)



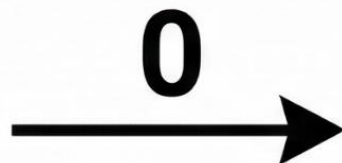
Um círculo simples =
Estado (Q - Quê)



Define todos os
estados possíveis
(ex: q_1, q_2).



Transição (δ - Delta)



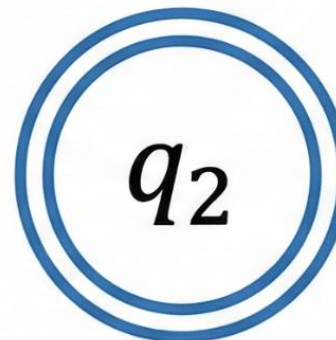
Uma seta com um '0'
em cima = **Transição (δ -
Delta)** por meio do
símbolo (Σ - Sigma)



Muda de estado com
base na entrada (ex:
de q_0 para q_1 ao ler 0)



Estado Final (F - Efe)



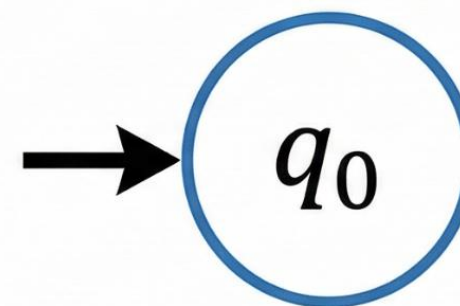
Um círculo com dois
contornos = **Estado
Final (F - Efe)**



Estado onde o autômato
'aceita' a cadeia (ex:
 $F = \{q_2\}$).



Início (q_0 - Quê zero)



Uma seta curta
apontando para o
primeiro círculo
= **Início (q_0 - Quê zero)**

Indica onde o
processamento começa
(ex: q_0).



Autômato Finito Determinístico (DFA)

1. Unicidade (O caminho é único)

Para cada estado e para cada símbolo do alfabeto, deve existir **exatamente uma** transição de saída.

2. Totalidade (Sem "buracos")

A função de transição deve ser total. Isso significa que **todo símbolo do alfabeto deve ter um destino definido** em todos os estados.

3. Proibição de Transições Vazias (ϵ)

Um DFA **nunca** pode mudar de estado sem consumir um caractere da entrada.

4. Alfabeto e Estados Finitos

Como o nome diz, tudo deve ser finito:

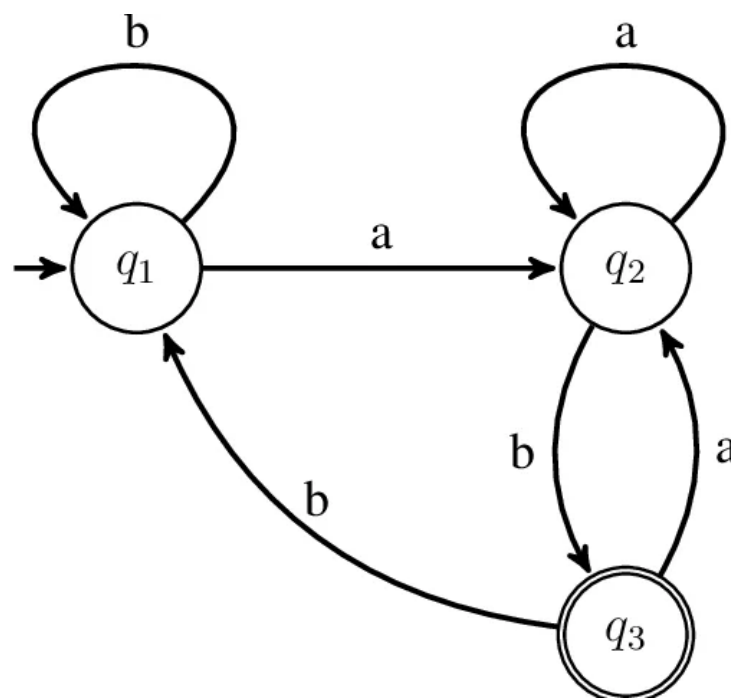
- O conjunto de estados (Q) deve ser finito.
- O alfabeto (Σ) deve ser um conjunto finito de símbolos.

5. Um Único Estado Inicial

Um DFA começa sempre em um único lugar. Você não pode ter duas setas de "entrada" apontando para estados diferentes no início do processo.

Por que este é um DFA?

1. **Completude (Totalidade):** Cada estado (q_1, q_2, q_3) possui exatamente uma transição de saída para **cada símbolo** do alfabeto (a e b). Não há "becos sem saída".
2. **Unicidade:** Não há ambiguidade. Ao ler um 'a' no estado q_1 , você só tem um destino possível (q_2). Não existem duas setas com a mesma letra saindo do mesmo lugar para destinos diferentes.
3. **Ausência de ϵ :** Não existem transições vazias ou espontâneas. O autômato só se move se ler um caractere.



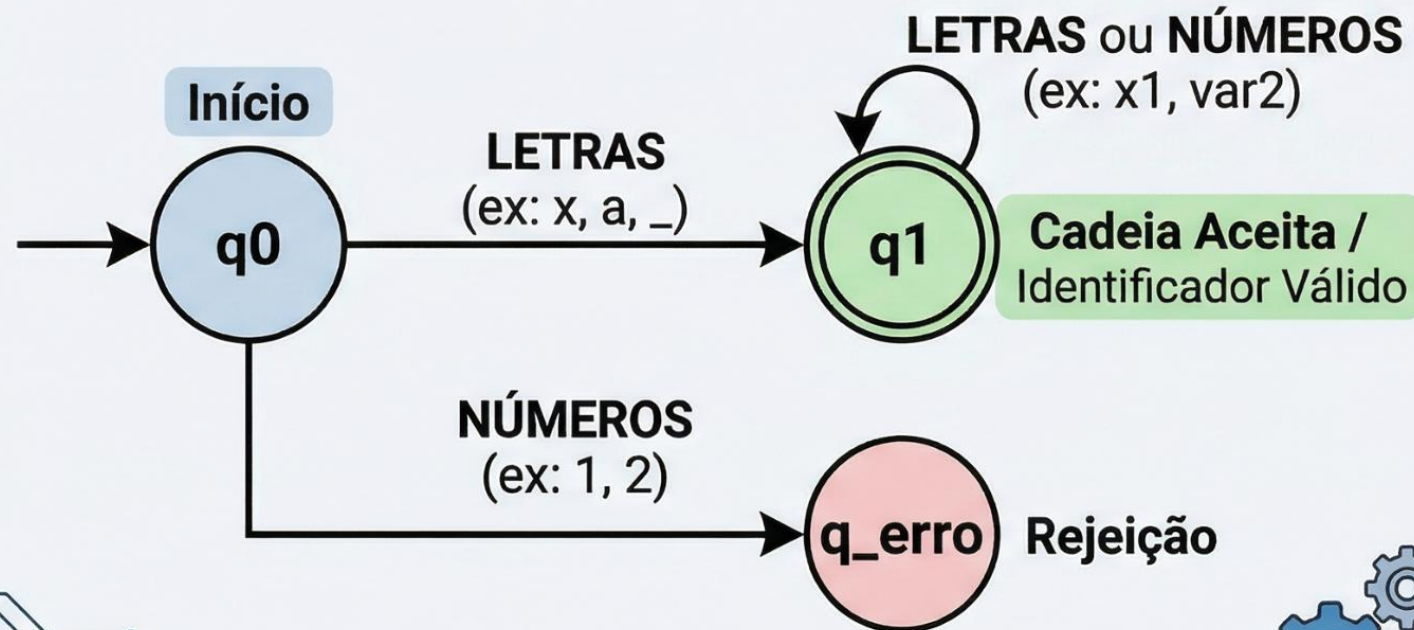
Exemplo Prático: Identificadores de Variáveis

Regra de Nomes de Variáveis

Um nome de variável **DEVE** começar com uma **LETRA** e **PODE** ser seguido por **LETRAS** ou **NÚMEROS**.

Autômato Finito Determinístico (AFD) para Validação

É Regular? Sim! Um AFD simples de 2 estados valida isso.



Por que Importa? O Papel do Compilador

```
code.cpp
1 #include <cstdlib>
2
3 int main() {
4     int x = 10;
5     if (x == 1) {
6         System.out.println("Valido", 1);
7     }
8 }
9
10 else {
11     int y = 0;
12     if (y > 0) {
13         System.out.println("Teste: ", " + 10);
14     }
15 }
```

✓ **Válido**
(Identificador Correto)

`int x1 = 10;`

`int 1x = 20;`

Compilador
(Análise Sintática)

✗ **Erro de Sintaxe:**
Identificador Inválido
(começa com número)

O compilador usa essa lógica exata de autômatos para **validar os nomes** das **suas variáveis**.

Informática Teórica

Aula 4

Autômato Finito Não-Determinístico (AFN)

Se no AFD tínhamos um caminho único e previsível, aqui apresentamos a ideia de **escolha e paralelismo**.

Definição Técnica

No Autômato Finito Não-Determinístico (AFN), a máquina pode ter várias opções de transição para o mesmo símbolo de entrada, ou até mesmo nenhuma.

Enquanto o AFD é uma função rígida, o AFN permite que o sistema "escolha" um caminho entre vários possíveis.

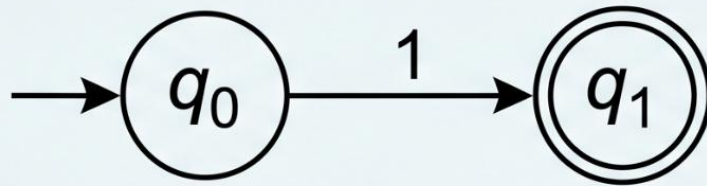
Definição Técnica

Segundo Sipser (2007), o não-determinismo pode ser visualizado como um tipo de computação paralela, onde a máquina se divide em várias cópias de si mesma para explorar todas as possibilidades simultaneamente.

INFORMÁTICA TEÓRICA: DETERMINISMO vs. NÃO DETERMINISMO

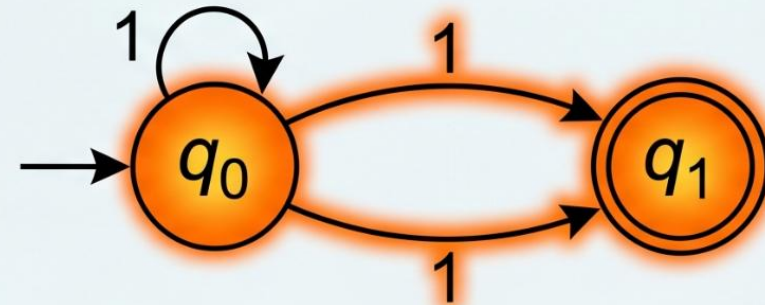
O Caminho das Escolhas: AFD e AFN

AFD: Lógica Determinística
(*Deterministic*)

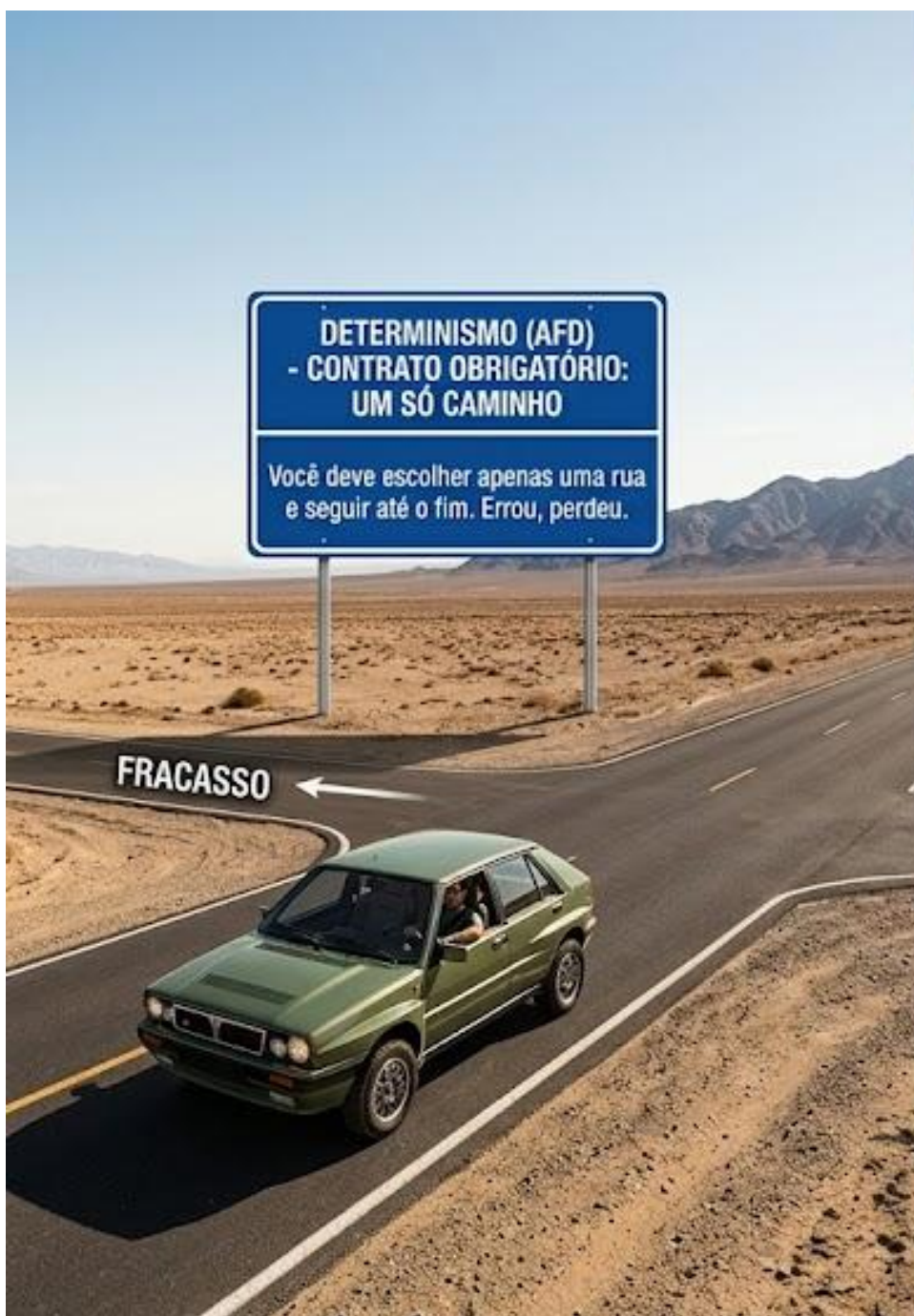


De q_0 , lê "1" e vai
Apenas para q_1 .

AFN: Lógica Não Determinística
(*Nondeterministic*)



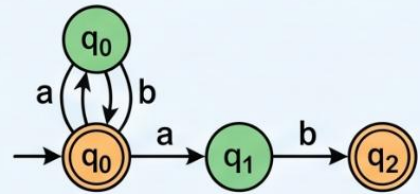
Ler "1" no AFN ramifica a execução.
A máquina "está" em $\{ q_0, q_1 \}$
simultaneamente!



POR QUE USAR AFN SE ELE NÃO EXISTE “NA VIDA REAL”?

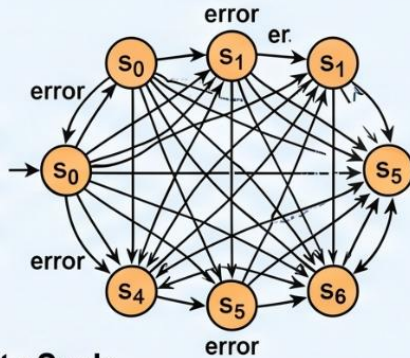
- 1 Simplicidade de Design:** É muito mais fácil desenhar um AFN[†] para problemas complexos do que um AFD.

AFN (Exemplo)



■ : Reconhece cadeias que terminam com “a”

AFD Equivalente



Simplicity Scale



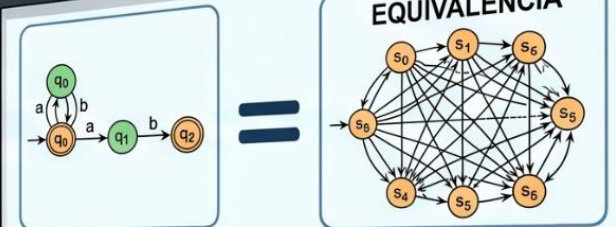
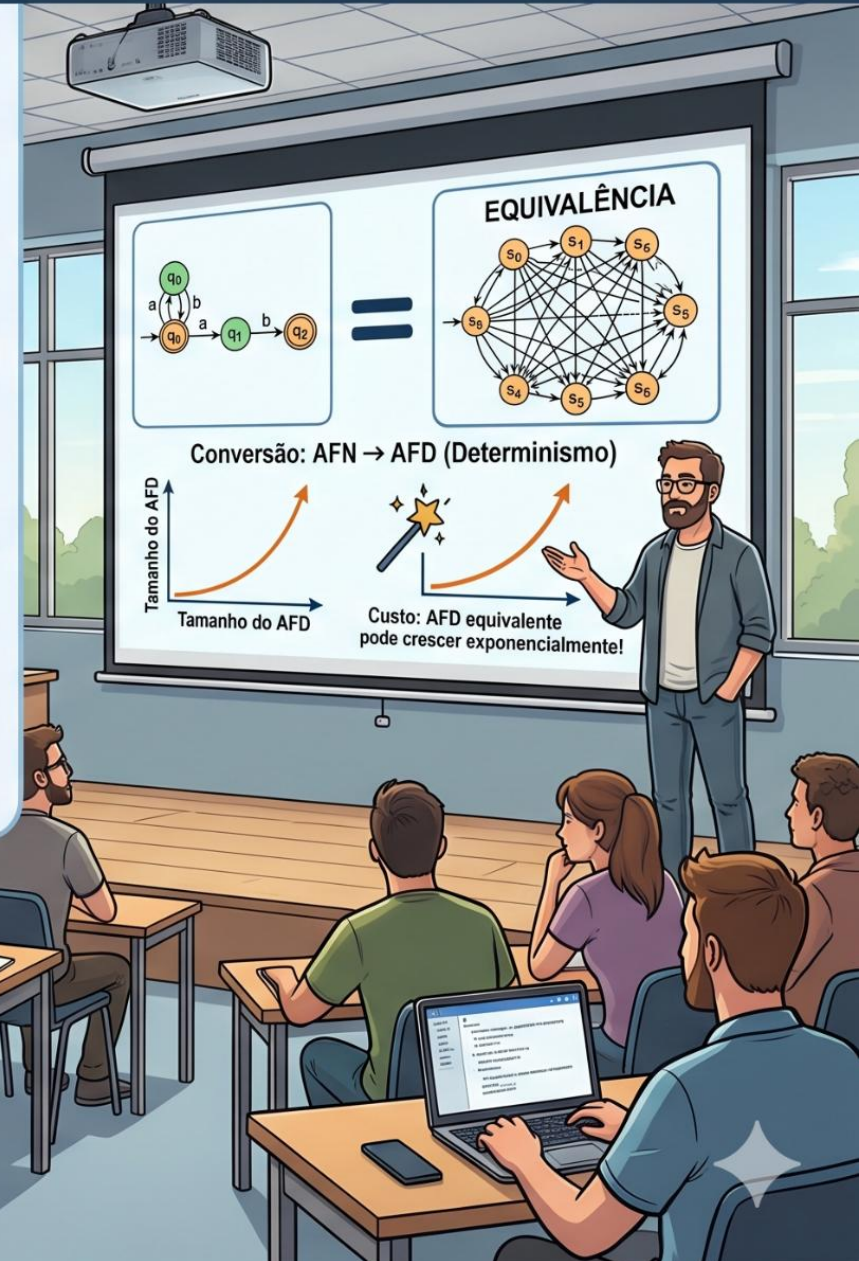
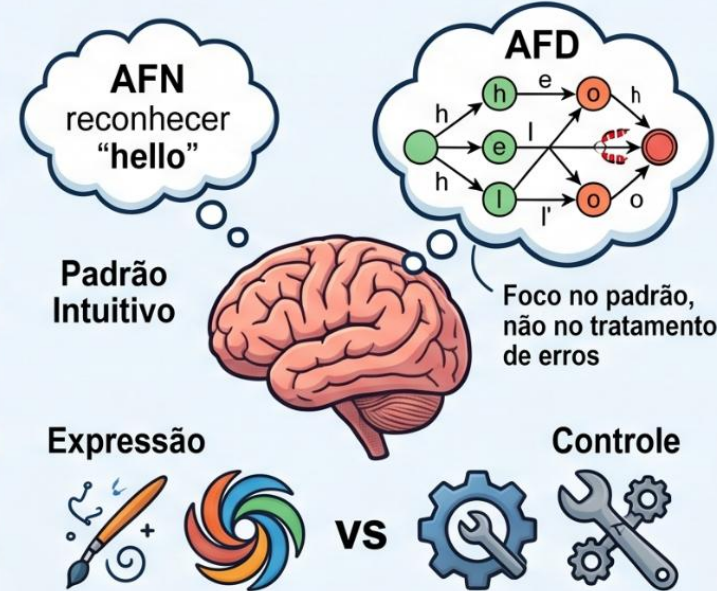
AFN — High

AFD — Holo

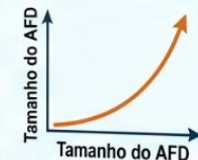


■ Custo de Design: Baixo ■ Custo de Design: Alto

- 2 Poder de Expressão:** O AFN permite focar no “que” o sistema deve reconhecer, sem se preocupar tanto com o “como” tratar cada erro imediatamente.



Conversão: AFN → AFD (Determinismo)



Custo: AFD equivalente pode crescer exponencialmente!

Transições Vazias (ϵ - Epsilon)

Definição Técnica

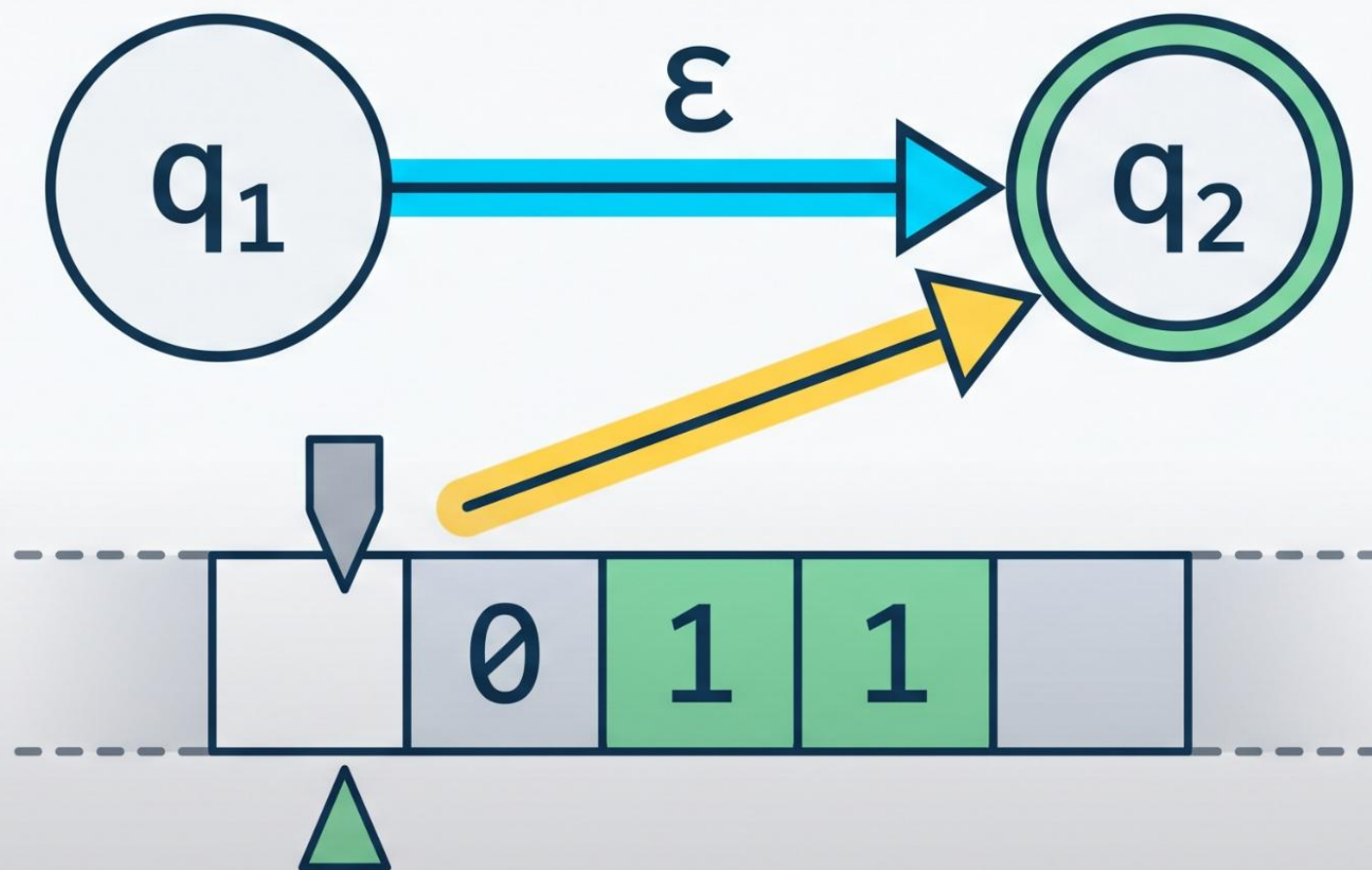
A transição ϵ permite que o autômato mude de estado **sem consumir nenhum símbolo** da fita de entrada.

No AFD, a máquina está "presa": ela só se move se ler algo. **No AFN, ela pode "escolher" mudar de estado espontaneamente.**

Conforme **Sipser (2007)**, as transições vazias são fundamentais para a flexibilidade do design, permitindo que a máquina entre em novos estados de busca "de graça".

Segundo **Hopcroft (2001)**, a inclusão do ϵ no alfabeto de transição define a capacidade da máquina de **reagir a uma "string vazia"**.

O Atalho Invisível



O Portal de Teletransporte: Analogia para Transições em Autômatos

TRANSIÇÃO NORMAL (USANDO SÍMBOLO)



É necessário o item (símbolo) correspondente para progredir.

TRANSIÇÃO ϵ (TELETRANSPORTE GRATUITO)

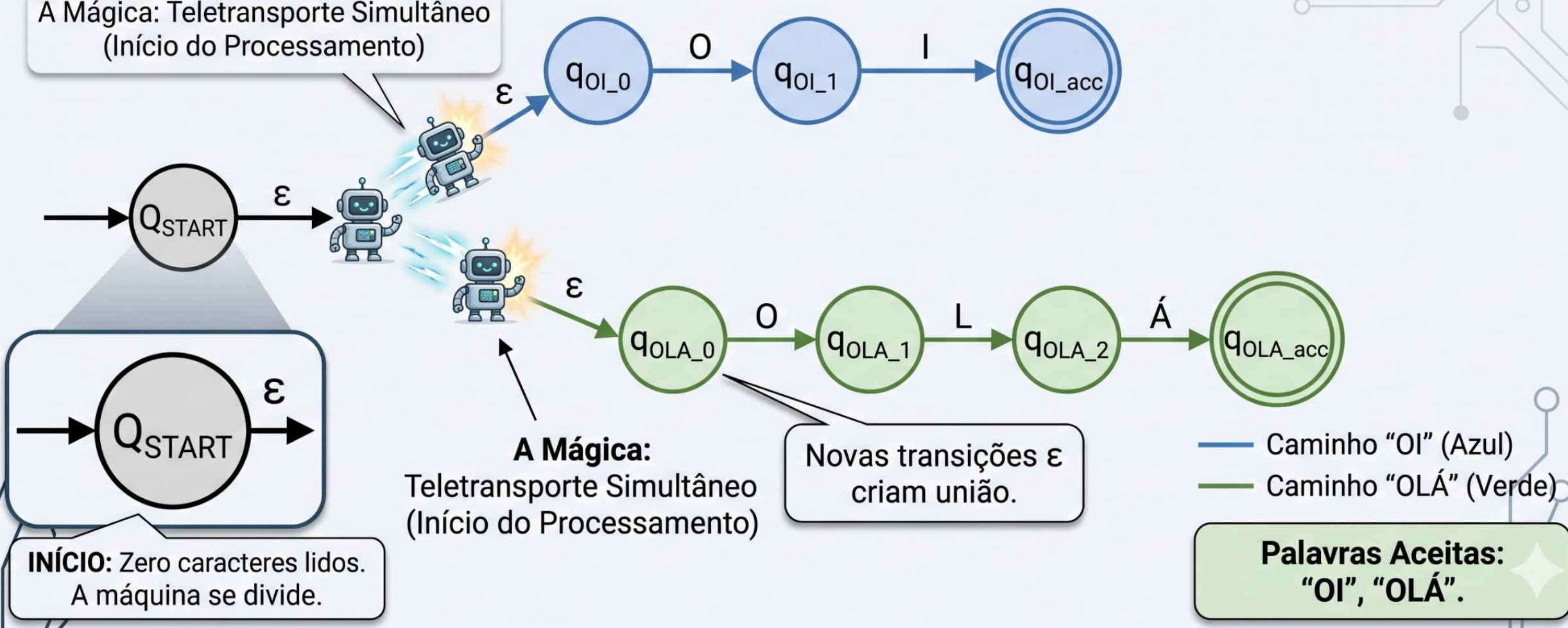


Um portal mágico que teletransporta instantaneamente para outro ponto, sem custo.

Exemplo Prático: Unindo Decisões

- Máquina que aceita “OI” ou “OLÁ” usando transições ϵ .

A Mágica: Teletransporte Simultâneo
(Início do Processamento)



A Definição Formal do AFN

1. Definição Técnica (A 5-upla Revisitada)

Assim como o AFD, o **AFN** é definido por uma 5-upla $M = (Q, \Sigma, \delta, q_0, F)$. A grande diferença está na **Função de Transição** (δ).

- Q (**Quê**): Conjunto finito de estados.
- Σ (**Sigma**): Alfabeto finito.
- q_0 (**Quê zero**): Estado inicial.
- F (**Efe**): Conjunto de estados de aceitação.
- δ (**Delta**): A função de transição agora é definida como:

- δ (**Delta**): A função de transição agora é definida como:

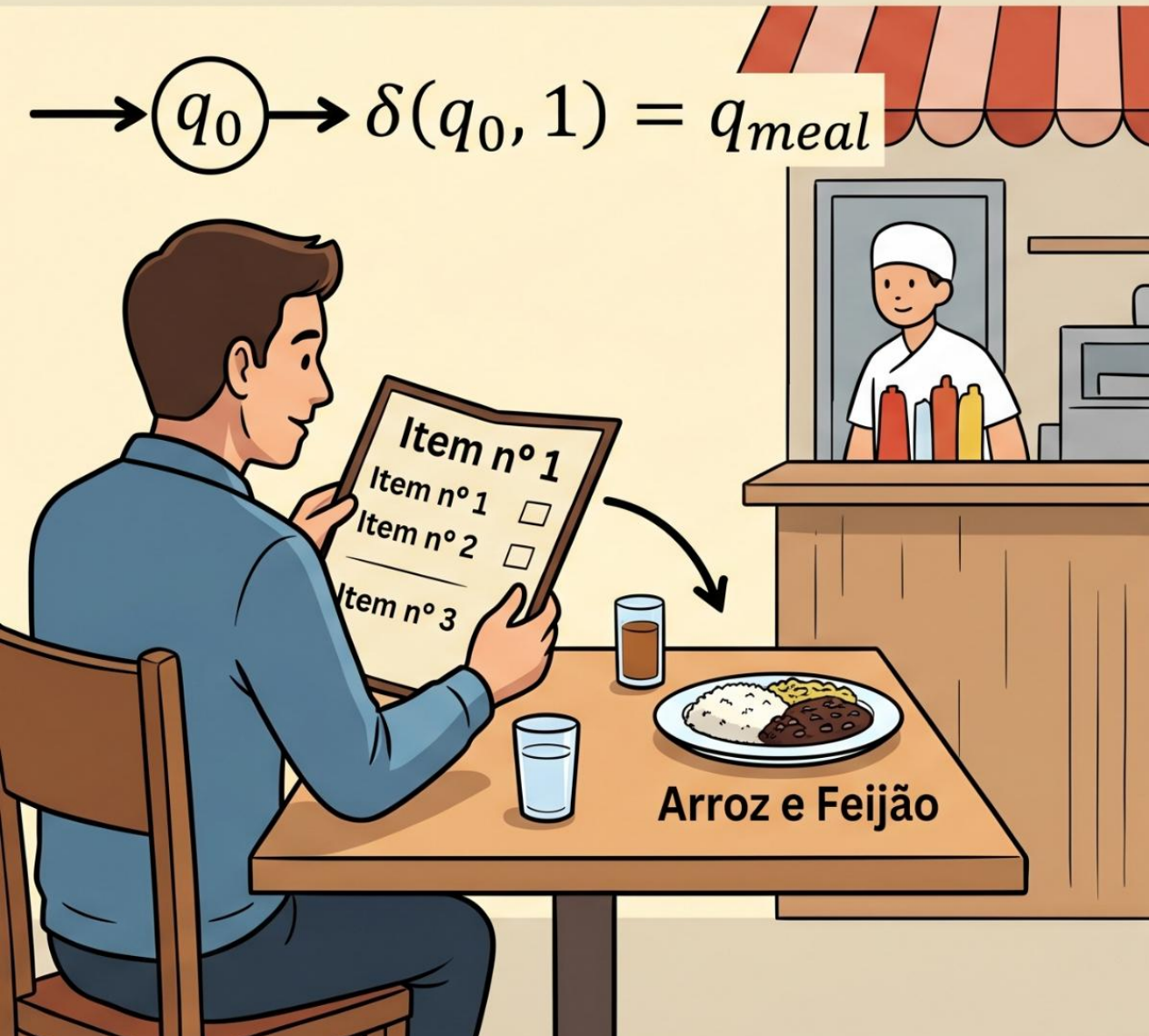
$$\delta : Q \times \Sigma_{\epsilon} \rightarrow \mathcal{P}(Q)$$

O que isso significa?

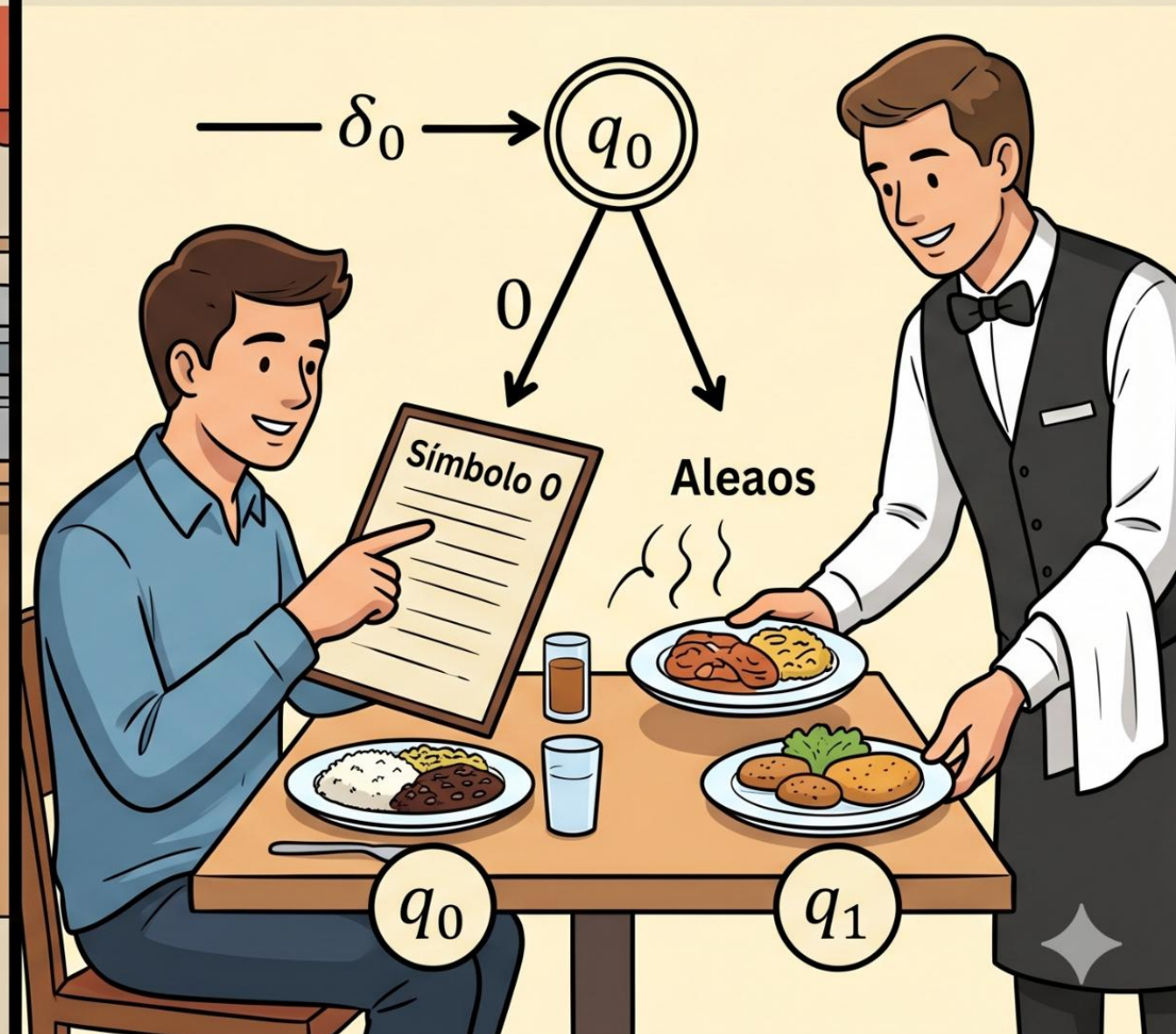
1. Σ_{ϵ} (**Sigma Epsilon**): O alfabeto agora inclui o vazio ($\Sigma \cup \{\epsilon\}$).
2. $\mathcal{P}(Q)$ (**Conjunto das Partes**): O resultado da transição não é mais um único estado, mas sim um **conjunto (uma lista)** de estados possíveis.

$\mathcal{P}(Q)$ diz que a resposta da função é uma "listinha" de estados. Se a lista tem 2 estados, a máquina se duplica. Se a lista é vazia, aquele caminho fecha.

AFD (Determinístico): O “Prato Feito”



AFN (Não-Determinístico): O “À La Carte” de Porções Duplas



INFORMÁTICA TEÓRICA: Exemplo Matemático Rápido - AFN

EXEMPLO MATEMÁTICO RÁPIDO: Autômato Finito Não-Determinístico (AFN)

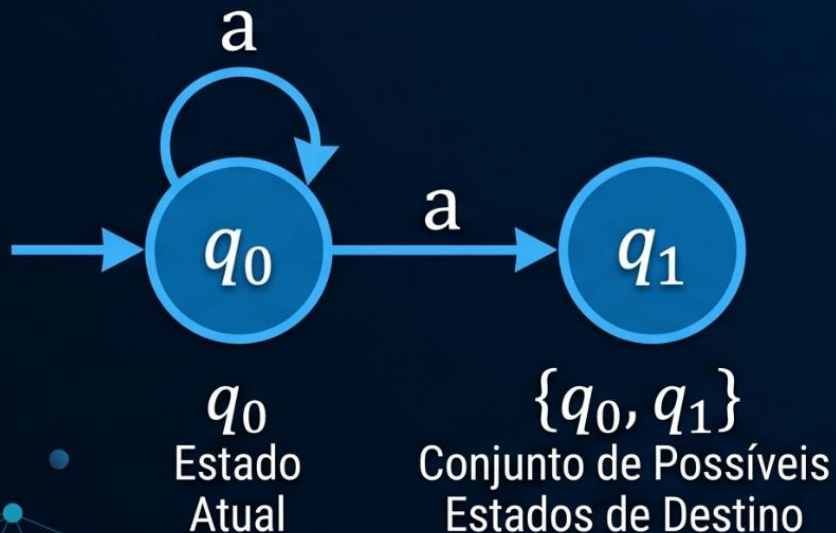
DEFINIÇÃO DA FUNÇÃO DE TRANSIÇÃO (δ)

CENÁRIO 1: Múltiplas Escolhas
(Não-Determinismo)

No estado q_0 , ao ler o símbolo 'a', o AFN pode:

- Ficar em q_0
- Ir para q_1

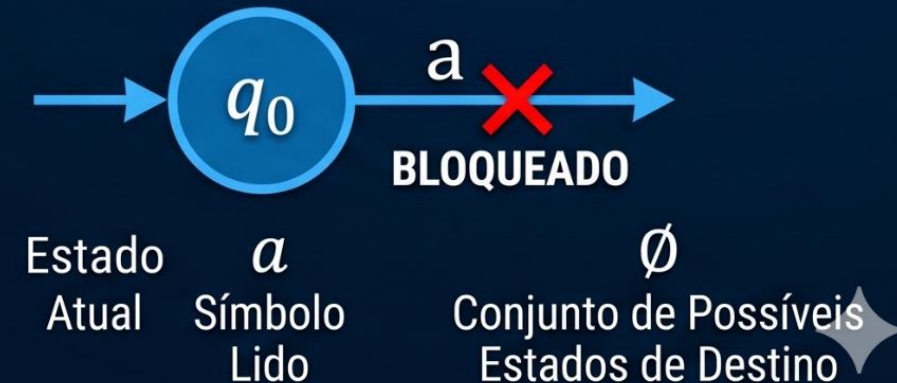
$$\delta(q_0, a) = \{q_0, q_1\}$$



CENÁRIO 2: Nenhuma Transição Definida
(Caminho Bloqueado)

Se no estado q_0 , ao ler 'a', o AFN não possui transição definida:

$$\delta(q_0, a) = \emptyset$$



O Critério de Aceitação no AFN

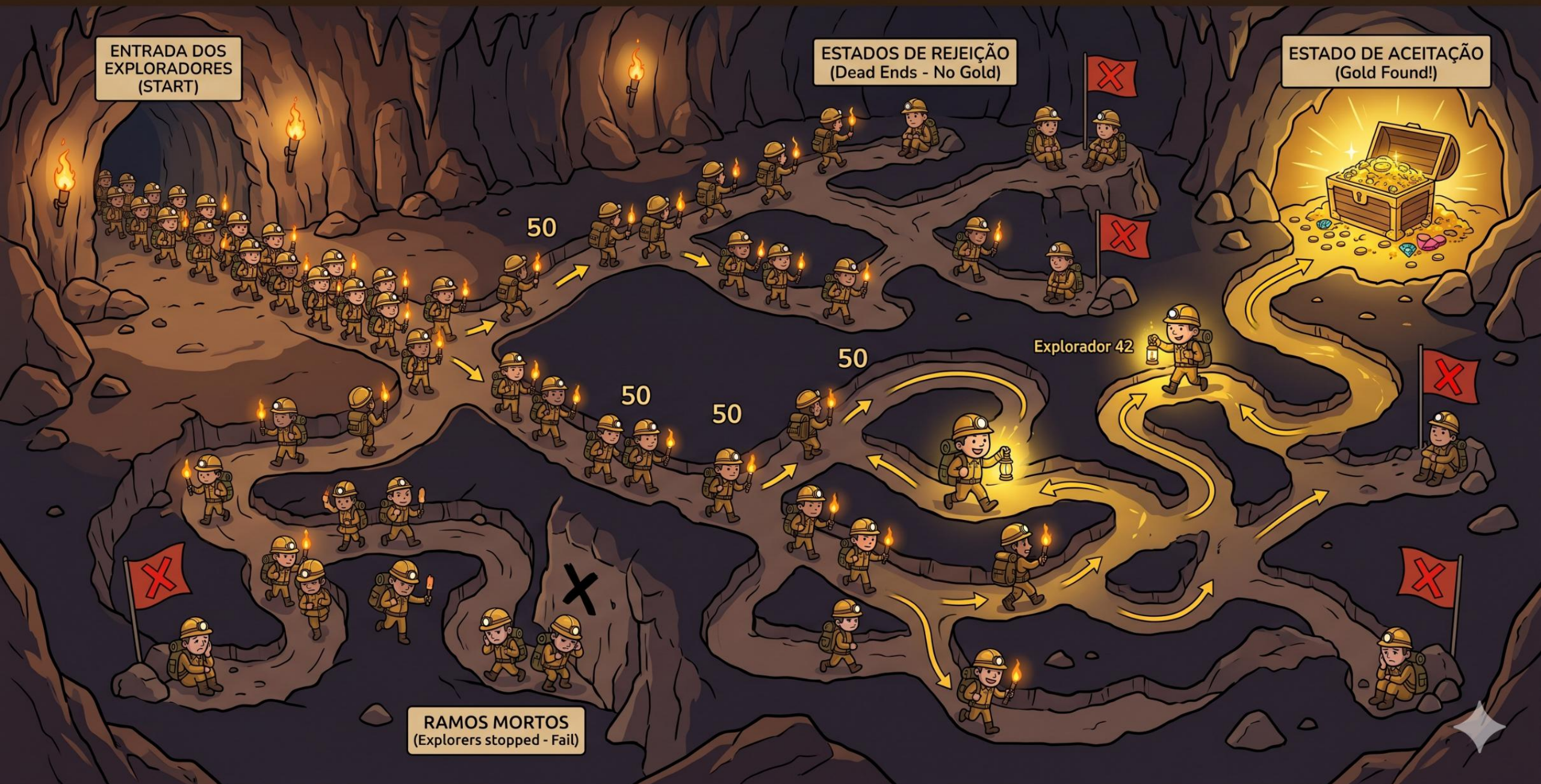
Se a máquina se divide em várias, qual delas vale?

1. Definição Técnica

Ao contrário do AFD, onde existe apenas um caminho possível, o processamento de um AFN gera uma **Árvore de Computação**.

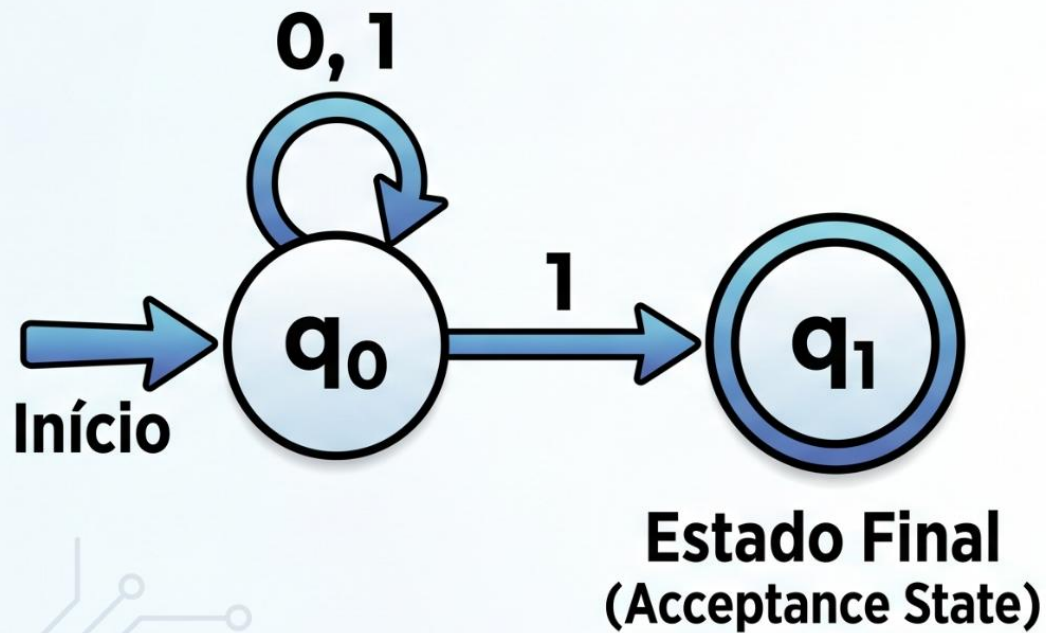
- **Regra de Aceitação:** Uma cadeia w é aceita pelo AFN se existir **pelo menos um** caminho (ramificação da árvore) que termine em um estado de aceitação (F) após a leitura de toda a entrada.
- **Morte de Ramos:** Se um ramo da computação tenta ler um símbolo para o qual não há transição definida ($\delta(q, a) = \emptyset$), esse ramo "morre" e é descartado.
- Segundo **Sipser (2007)**, o AFN pode ser visto como uma máquina que sempre "escolhe corretamente" o caminho para a aceitação, se ele existir.

ANALOGIA: A CORRIDA DO OURO (Informática Teórica)



EXEMPLO PRÁTICO DE AFN: 'TERMINA COM 1'

1. O DIAGRAMA DO AFN (NFA DIAGRAM)



2. RASTREAMENTO DA CADEIA '01'

Cadeia de Entrada: 0 1

Início: A máquina está em q_0 q_0 Instância 1

Lê 0: q_0 Continua em q_0

Lê 1: q_0 q_1 **Divisão!**

Instância 1a: Uma cópia fica em q_0

Instância 1b: Uma cópia vai para q_1

Fim da Cadeia
(End of String)

✗ REJEITA

✓ ACEITA

Como a cópia em q_1 é de aceitação, a palavra é ACEITA.
(As the copy in q_1 accepts, the word is ACCEPTED.)

Equivalência entre AFN e AFD

1. Definição Técnica

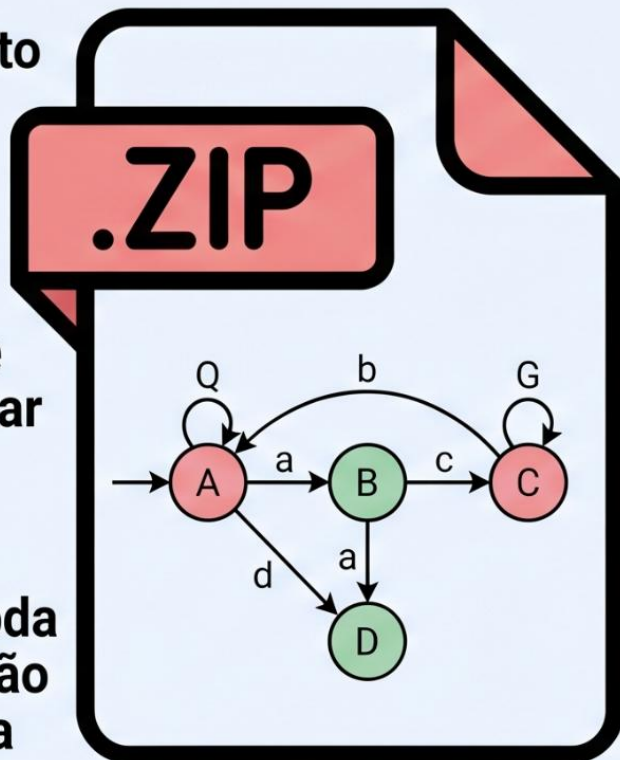
Embora o AFN pareça mais potente devido à sua capacidade de "multiplicação", ele reconhece a mesma classe de linguagens que o AFD: as **Linguagens Regulares**.

- **Teorema da Equivalência:** Para cada AFN, existe um AFD equivalente que aceita exatamente a mesma linguagem.
- **O Algoritmo:** Utilizamos a **Construção de Subconjuntos** (ou Construção por Conjunto das Partes). A ideia é que cada estado do novo AFD represente um *conjunto* de estados do AFN original.
- **Explosão de Estados:** Se um AFN tem n estados, o AFD equivalente pode ter até 2^n estados. É por isso que preferimos projetar em AFN (é muito mais compacto!).

Analogia: O Arquivo ZIP para Informática Teórica (AFN e AFD)

AFN (ARQUIVO .ZIP)

- Compacto



- Fácil de transportar

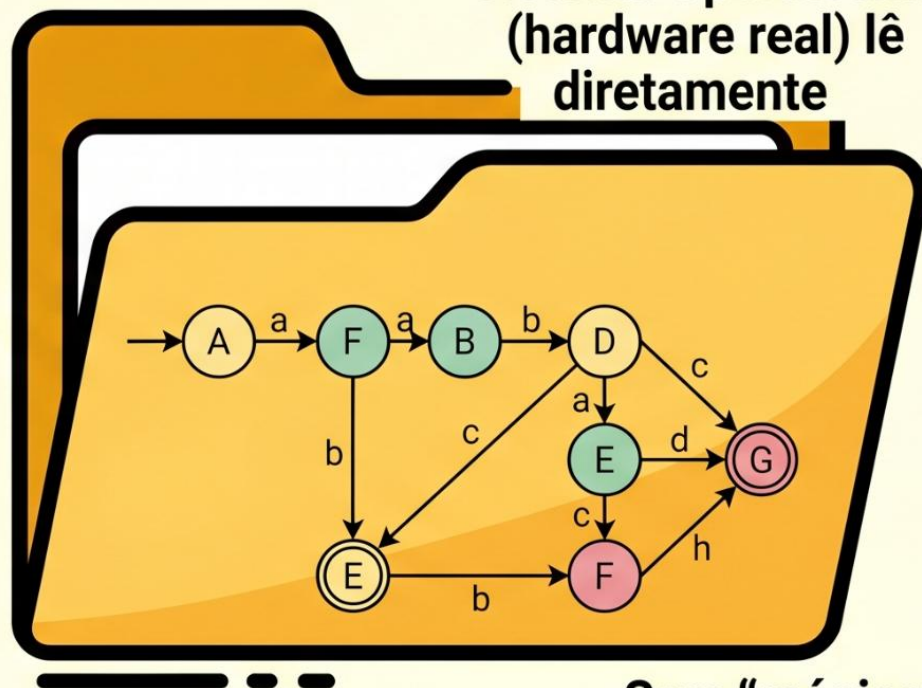
- Contém toda a informação de forma resumida

PROCESSO DE
DESCOMPACTAÇÃO
/ CONVERSÃO

AFD (PASTA DESCOMPACTADA)

- Ocupa mais espaço

- Único formato que o "Sistema Operacional" (hardware real) lê diretamente



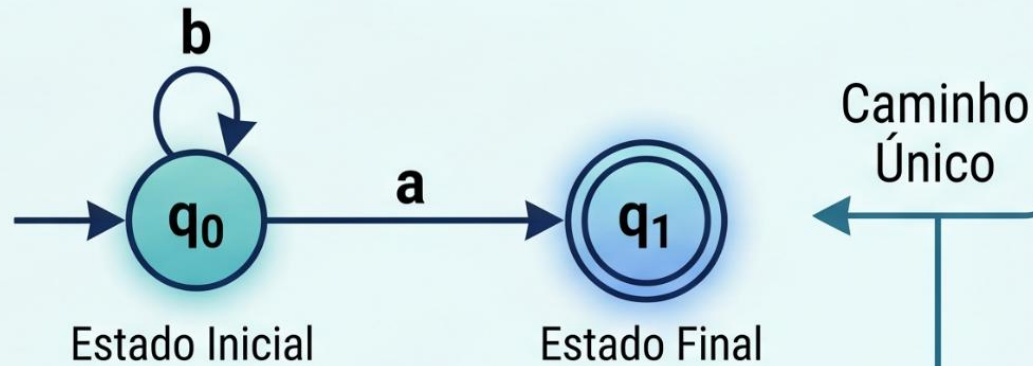
- Sem "mágica" ou suposições

CONCLUSÃO: O conteúdo (a linguagem) é rigorosamente o mesmo; o que muda é apenas a forma como a informação está organizada.



Ilustração Comparativa: AFD vs. AFN

Autômato Finito Determinístico (AFD)



Caminho Único e Previsível

1 Transição por Símbolo e Estado

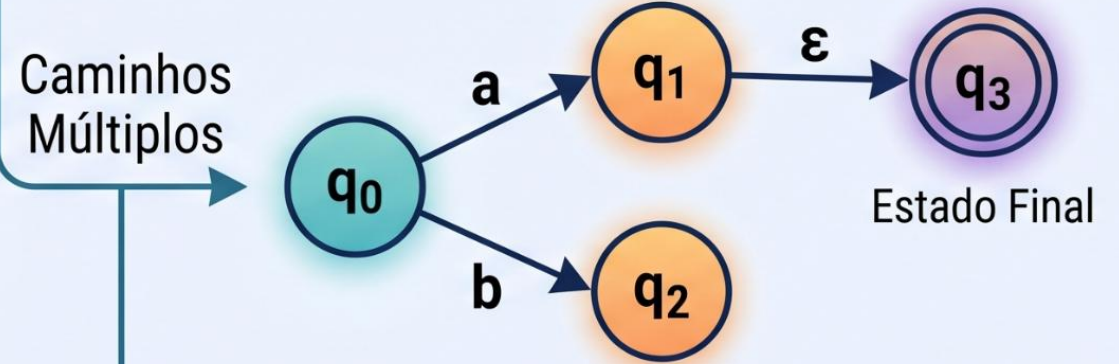


Linguagens Regulares

Estado/Símbolo	a	b
q ₀	q ₁	q ₀
q ₁	q ₁	q ₁



Autômato Finito Não Determinístico (AFN)



Múltiplos Caminhos Possíveis

0, 1 ou mais Transições por Símbolo
Transições Vazias (ϵ) Permitidas



Linguagens Regulares

–o mesmo tipo de linguagem

Estado/Símbolo	a	b	ϵ
q ₀	{q ₁ , q ₂ }	{q ₁ }	{ }



O Fluxo de Trabalho na Computação

O Fluxo de Trabalho na Computação

Na prática, você não escolhe apenas um. O processo padrão na Ciência da Computação segue esta ordem:

1. **Especificação:** Você escreve uma **Expressão Regular** (ex: `[a-z]+[0-9]*`).
2. **Modelagem:** Você converte essa expressão em um **AFN** (porque é fácil e direto).
3. **Otimização:** Você usa um algoritmo (como a **Construção de Subconjuntos**) para transformar esse AFN em um **AFD**.
4. **Minimização:** Você simplifica o AFD para que ele tenha o menor número de estados possível.
5. **Execução:** O computador roda o **AFD Minimizado** para processar os dados com velocidade máxima.

Por que fazer um AFN (Não Determinístico)?

O AFN é a sua ferramenta de **modelagem e design**. Ele é pensado para facilitar a vida do ser humano que está projetando a lógica.

- Simplicidade de Design
- Abstração e Intuição
- Tamanho Compacto
- Transições Vazias (ϵ)

Por que fazer um AFD (Determinístico)?

O AFD é a sua ferramenta de **implementação e execução**. Ele é pensado para a máquina.

Eficiência Computacional

Fácil de Programar: Implementar um AFD em código é simples; basta uma matriz de transição ou um bloco switch/case dentro de um laço while.

Aplicações Reais: Compiladores usam AFDs na fase de **Análise Léxica**.

Resumo Comparativo de Uso

Objetivo	Use AFN	Use AFD
Criar uma lógica do zero	✓ Sim	✗ Difícil
Representar Regex	✓ Ideal	✗ Complexo
Rodar em um software	✗ Lento	✓ Ideal
Economizar memória na execução	✗ Ruim	✓ Excelente

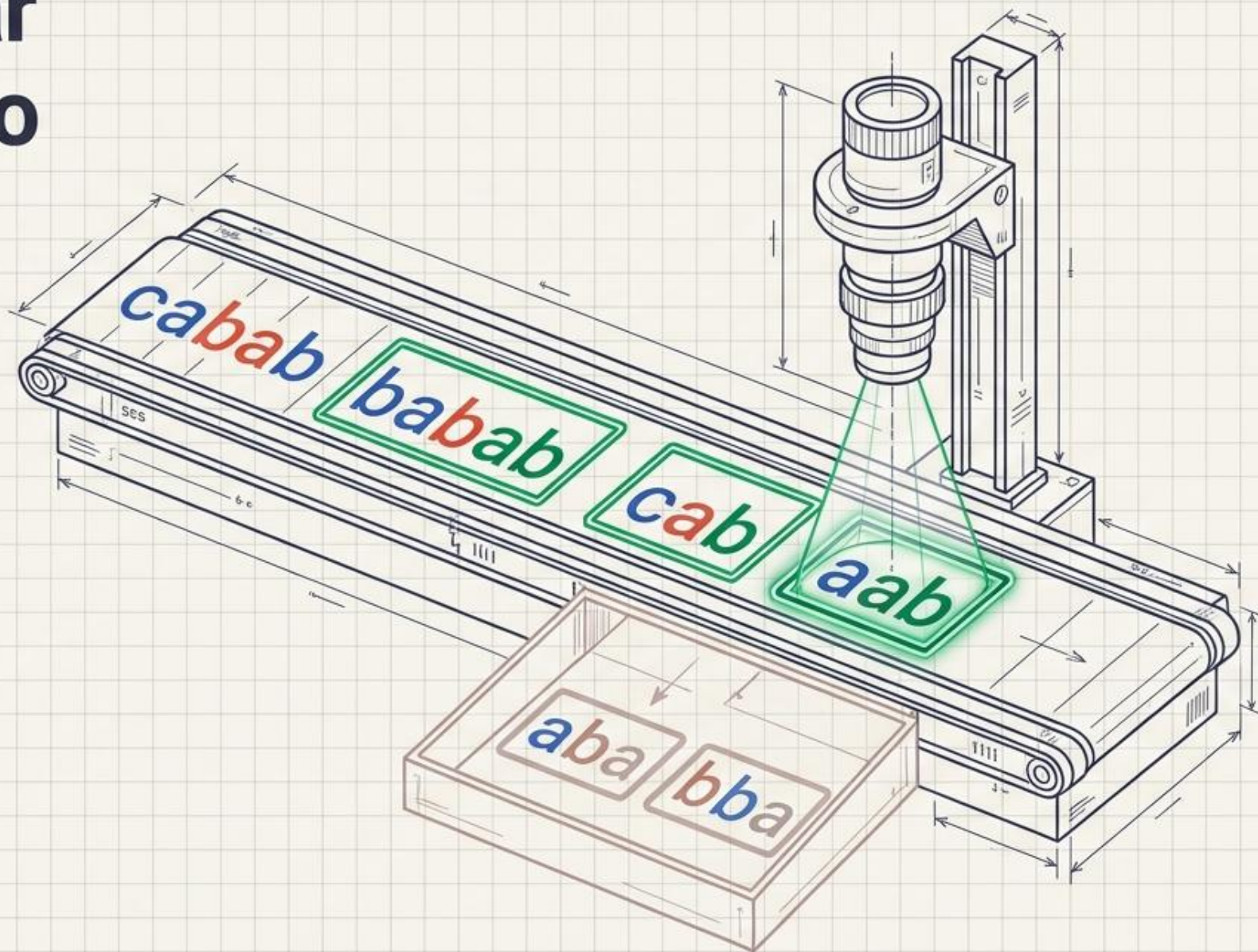
Regex é um conjunto de códigos que define um "molde" ou padrão para encontrar, validar e manipular textos automaticamente.

Ler e analisar Exemplo Prático

O Desafio: Isolar o Sufixo Perfeito

Objetivo: Criar um sistema capaz de identificar se uma cadeia de caracteres termina exatamente com o sufixo 'ab'.

Regra de Ouro: O tamanho da palavra ou o que vem antes não importa. Apenas o final absoluto é avaliado.



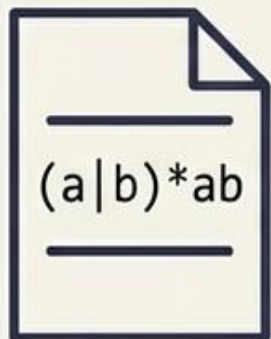
A Linha de Montagem da Lógica

Uma jornada estruturada: traduzimos a intenção humana num formato abstrato, refinamos a lógica para eliminar a ambiguidade e, finalmente, geramos código de alta performance.

01

O Contrato

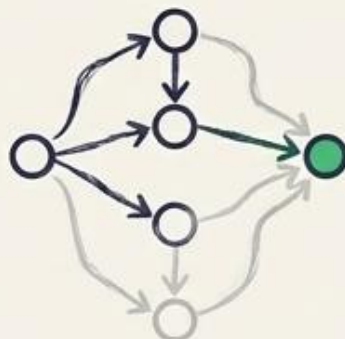
Expressão Regular
(ER)



02

O Esboço

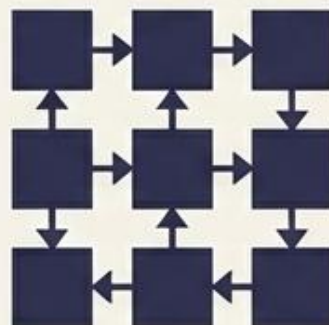
Autómato Não-Determinístico (AFN)



03

A Precisão

Autómato Determinístico (AFD)



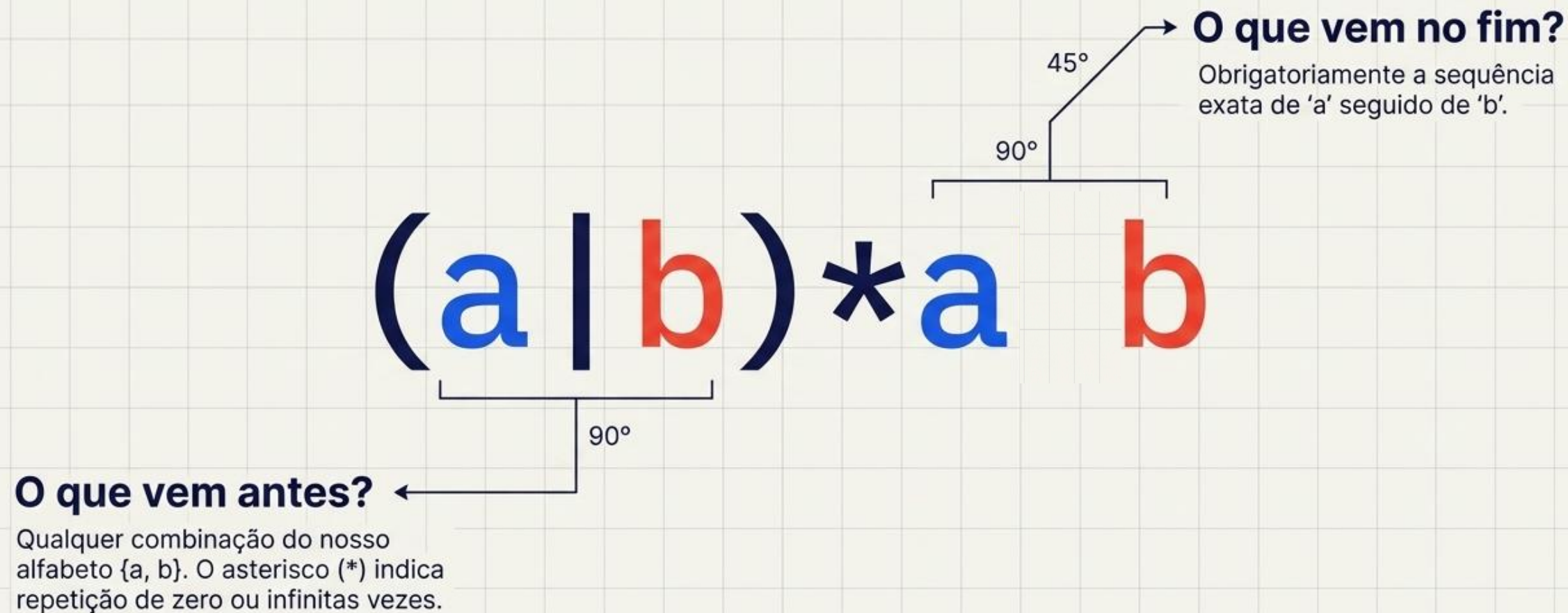
04

A Máquina

Código de Execução

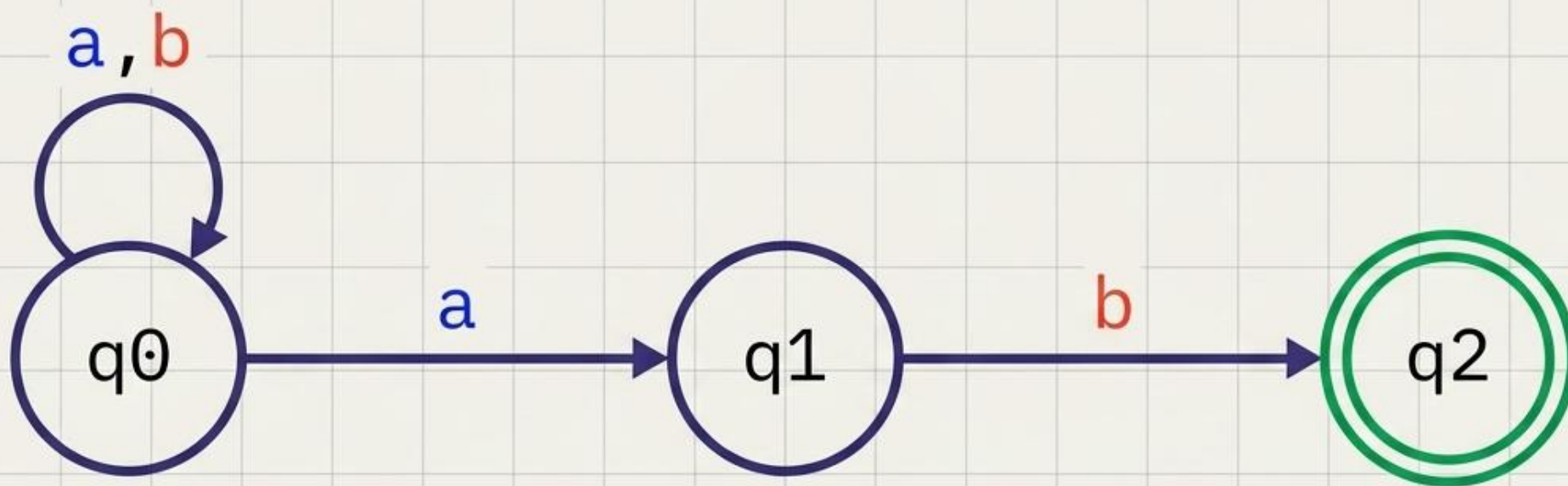
```
bool matches(string s) {  
    state = q0;  
    return true;  
    ...  
    return false;  
}
```

Passo 1: A Expressão Regular (O Contrato)



A Expressão Regular é a forma mais compacta de formalizar o nosso padrão.

Passo 2: O AFN (O Esboço Intuitivo)



A Intuição

O Autômato Não-Determinístico (AFN) pode tomar decisões simultâneas perante um único caráter.

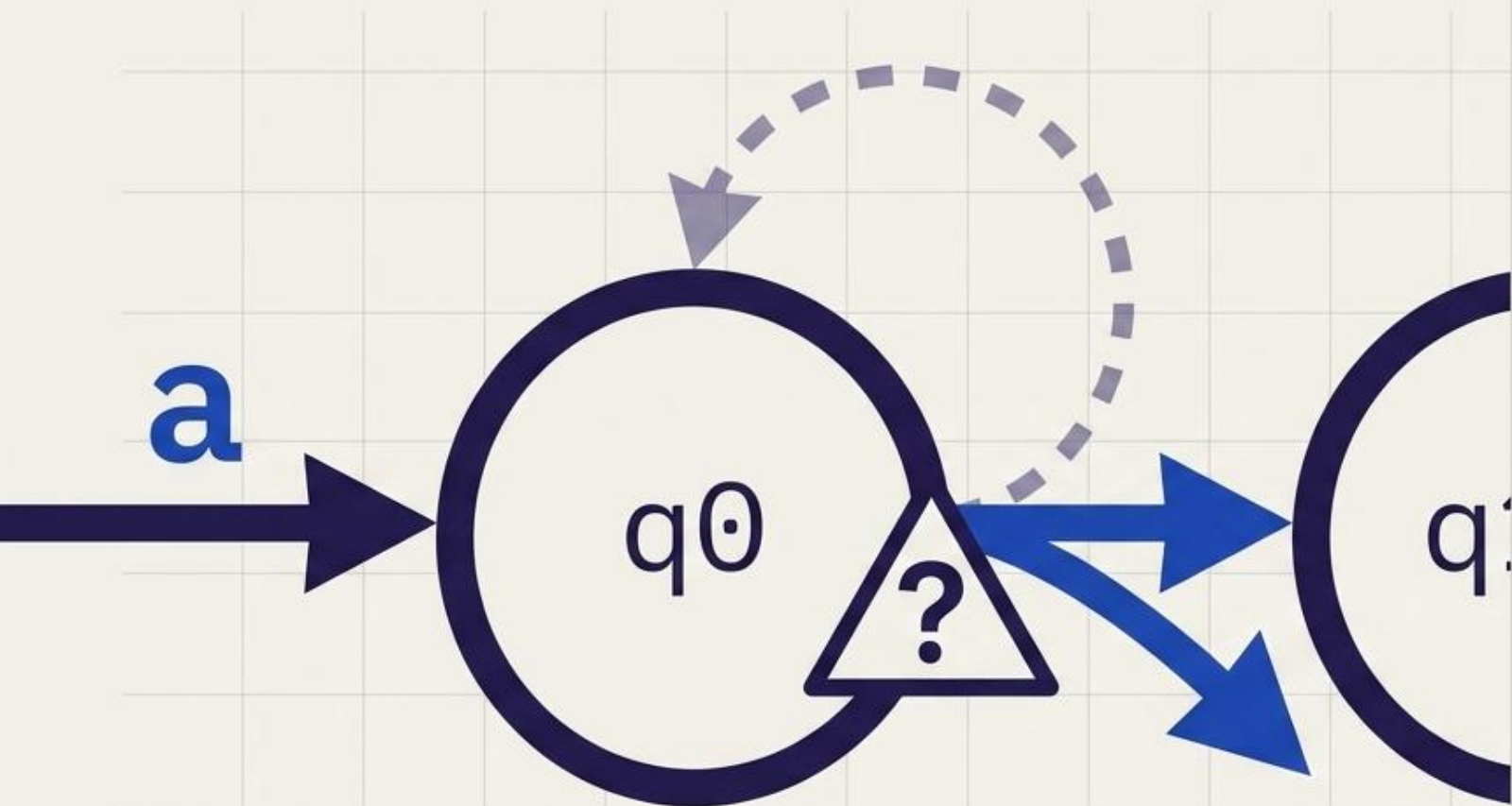
A Espera

O sistema fica preso no laço do estado q0 a ler caracteres indefinidamente até encontrar um padrão útil.

A Aposta

Ao ler um 'a', o sistema pode continuar a esperar em q0 OU apostar que é o início do sufixo e avançar para q1.

0 Paradoxo da Aposta



0 Problema Computacional:

Um processador mecânico não consegue 'apostar' ou existir em dois estados ao mesmo tempo.

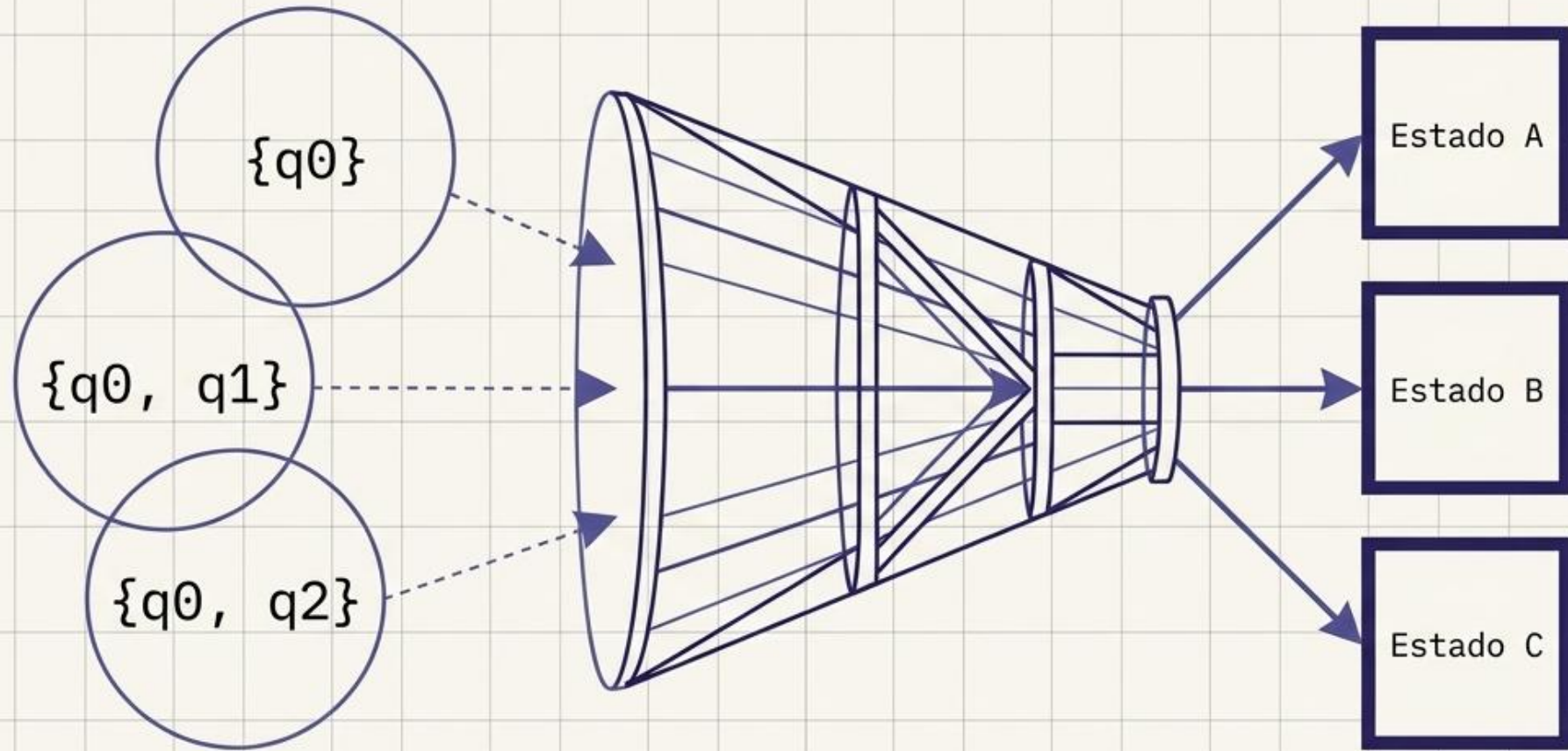
A Falha de Realidade:

Se ler um 'a' no estado q_0 , o AFN bifurca a sua realidade em dois caminhos divergentes simultâneos.

A Necessidade:

Precisamos de eliminar a adivinhação. O computador necessita de saber exatamente qual o único caminho absoluto a seguir.

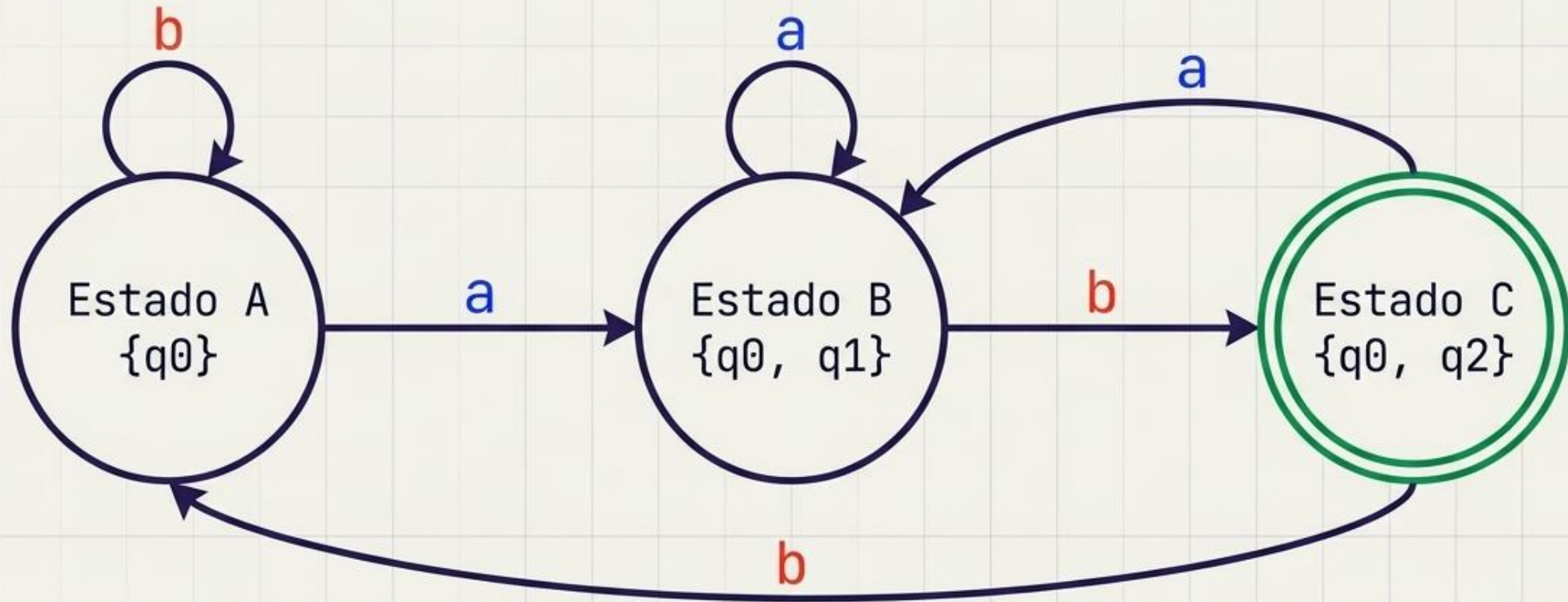
Passo 3: A Construção de Subconjuntos



A Engenharia de Precisão:

Para converter o AFN num Autômato Determinístico (AFD), agrupamos todas as posições onde a máquina poderia estar simultaneamente num único e impenetrável novo estado sólido. Agrupar incertezas cria certezas absolutas.

○ Grafo AFD (Engenharia de Precisão)



Sem Apostas:

Cada estado tem agora exatamente uma saída obrigatória para 'a' e uma saída obrigatória para 'b'. A ambiguidade foi matematicamente erradicada.

A Quebra de Padrão:

Se a máquina ler 'b' no Estado C, percebe que a sequência falhou e regressa instantaneamente à estaca zero (Estado A) sem hesitar.

1. O Problema da Ambiguidade

No AFN, o não-determinismo permite que você esteja em "vários lugares ao mesmo tempo". Se você simplificar o estado $\{q_0, q_1\}$ para apenas q_1 no seu AFD, você estaria **jogando informação fora**.

- Se você fosse apenas para o estado q_1 , você esqueceria que ainda existe a possibilidade de estar no estado q_0 (o laço inicial).
- Se vier um segundo 'a', o seu autômato "morreria" porque q_1 sozinho não sabe lidar com um novo 'a'.

Ao manter o conjunto $\{q_0, q_1\}$, o AFD diz: *"Eu sei que recebi um 'a' e estou pronto para o 'b' (por causa do q_1), mas também sei que posso receber outro 'a' e continuar tentando (por causa do q_0)"*.

2. O Conceito de "Estado Composto"

No AFD, cada estado é, na verdade, uma **combinação de possibilidades** do AFN original.

Imagine que o AFN é um mapa com vários caminhos e o AFD é um GPS que mostra todas as rotas possíveis simultaneamente:

- **Estado Único no AFD:** Representa uma **bolha** que engloba todos os estados onde o AFN poderia estar.
- **O Nome do Estado:** Batizar o estado como $\{q_0, q_1\}$ é apenas uma convenção didática. Matematicamente, esse conjunto **é** um único estado. Poderíamos chamá-lo de "Estado X", mas $\{q_0, q_1\}$ ajuda o projetista a saber de onde ele veio.

3. A Explosão de Estados (O Teorema)

Isso explica por que um AFD costuma ser maior que um AFN. Como o AFD precisa mapear todas as combinações possíveis de estados do AFN, o número máximo de estados que um AFD pode ter é 2^n (onde n é o número de estados do AFN).

Exemplo: Se um AFN tem 3 estados (q_0, q_1, q_2), o AFD correspondente poderia ter até 8 estados (2^3):

$\emptyset, \{q_0\}, \{q_1\}, \{q_2\}, \{q_0, q_1\}, \{q_0, q_2\}, \{q_1, q_2\}, \{q_0, q_1, q_2\}.$

Resumo Didático

O AFD não tem estados simples porque ele é um "**Simulador de AFN**". Para simular algo que pode estar em dois lugares ao mesmo tempo, o simulador precisa de um estado especial que represente essa "dualidade".

Se usássemos apenas q_1 , o autômato seria **incompleto**. Se usássemos apenas q_0 , ele seria **cego** para o padrão que queremos reconhecer. A união $\{q_0, q_1\}$ é o que o torna **determinístico e funcional**.

Conseguiu visualizar por que o AFD precisa dessa "memória combinada" para não se perder no processamento?

Síntese de Arquiteturas: AFN vs. AFD

	AFN (O Esboço)	AFD (A Máquina)
Paradigma	Aposta e Intuição	Rastreamento de Precisão
Mecânica	Pode ter múltiplas escolhas para a mesma entrada de dados.	Caminhos únicos e absolutos. Comportamento estritamente determinístico.
Computação	Difícil e ineficiente para o processador executar diretamente.	Execução imediata, puramente mecânica e de altíssima velocidade.
Design	Mais fácil e rápido para humanos desenharem mentalmente.	Exige o cálculo matemático rigoroso de subconjuntos para ser construído.

Passo 4: O Código (A Máquina em Ação)

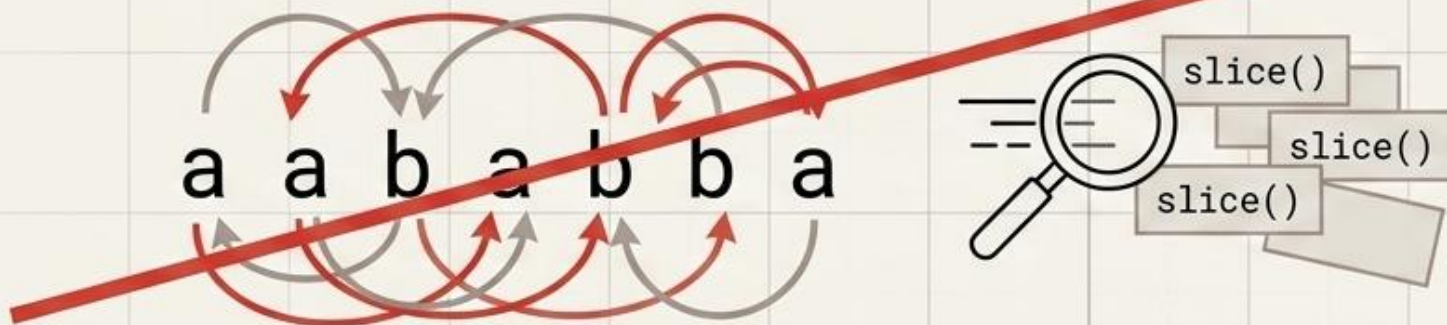
O AFD é puramente mecânico. Traduzimos o grafo desenhado numa simples matriz de regras, onde o estado atual e o carácter lido ditam o salto no código.

State Transition Matrix		
	Entrada 'a'	Entrada 'b'
Estado A	Estado B	Estado A
Estado B	Estado B	Estado C
Estado C	Estado B	Estado A

```
switch (estado_atual) {  
  case A:  
    if (c == 'a') estado_atual = B;  
    else estado_atual = A;  
    break;  
  case B:  
    if (c == 'a') estado_atual = B;  
    else estado_atual = C;  
    break;  
  case C:  
    if (c == 'a') estado_atual = B;  
    else estado_atual = A;  
    break;  
}
```

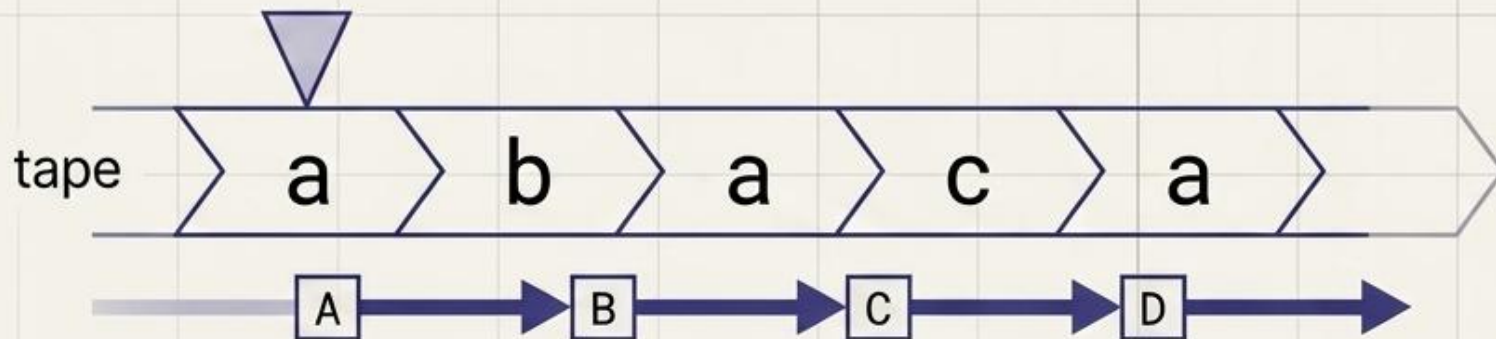
O Segredo da Performance

Abordagem Tradicional



Buscas complexas, funções pesadas na memória e loops aninhados que atrasam o processador.

Método AFD



O computador lê apenas um caráter de cada vez, pulando de estado em estado numa única passagem.

É esta arquitetura invisível que torna os analisadores léxicos e os motores de pesquisa incrivelmente rápidos.